

## Full Length Article

# LS-PRISM: A layer-selective pruning method via low-rank approximation and sparsification for efficient large language model compression

Renshuai Tao<sup>a</sup>, Hairong Chen<sup>a</sup>, Yuzhe Guo<sup>a</sup>, Jiakai Wang<sup>b</sup>, Boying Wang<sup>c</sup>,  
Rongrong Ni<sup>a,\*</sup>, Yao Zhao<sup>a</sup>

<sup>a</sup> Beijing Jiaotong University, Beijing, 100044, China

<sup>b</sup> Zhongguancun Laboratory, Beijing, 100094, China

<sup>c</sup> North University of China, Taiyuan, 030051, China

## ARTICLE INFO

## Keywords:

Large language models (LLMs)

Model compression

Low-rank approximation

Unstructured pruning

Sparsification

## ABSTRACT

Large Language Models (LLMs) have significantly advanced natural language processing (NLP), establishing new benchmarks across a wide range of tasks. However, their large parameter sizes present challenges for deployment on resource-constrained devices. Current compression techniques often treat all layers uniformly, disregarding the operational differences across layers, which can lead to varying effects on performance. In this paper, we introduce a novel method named Layer-Selective Pruning via low-Rank Approximation and Sparsification Method (LS-PRISM), efficiently compressing LLMs while preserving their performance on key NLP benchmarks, such as BoolQ, RTE, and ARC-Challenge. LS-PRISM dynamically applies low-rank approximation to selected matrices within each model layer based on their impact on accuracy and loss, with ranks adaptively determined using Dynamic Rank Selection method, retaining approximations that improve performance and leaving others unaltered. Additionally, we employ unstructured pruning on the remaining matrices to further sparsify the model, followed by optional fine-tuning using LoRA to recover performance loss. Experimental results demonstrate that LS-PRISM achieves significant reductions in both parameter count and storage, with minimal degradation in accuracy. Specifically, for a 2.5B parameter LLM, we achieve up to a 12% reduction in parameters, while maintaining performance comparable to the original model. We also explore the method's applicability to even smaller models, and discuss the observed performance differences. LS-PRISM offers a scalable and effective solution for compressing LLMs in resource-constrained environments.

## 1. Introduction

Deep learning has achieved remarkable success across a wide range of tasks (Gao et al., 2023; Li et al., 2023b; Liu et al., 2024b; Ma et al., 2023; Qi et al., 2024a; Sun et al., 2022; Tao et al., 2022; Wang et al., 2024a), and its advancements have significantly contributed to the revolution of natural language processing (NLP), particularly with the development of large language models (LLMs). These models have achieved outstanding performance in key areas such as language comprehension, text generation, and machine translation (Mi et al., 2022; Qi et al., 2024b; Wu et al., 2024). Notable models like GPT-3 (Brown, 2020), BERT (Devlin, 2018), and their successors have consistently demonstrated a correlation between increasing model size and enhanced capabilities. This trend underscores a critical paradigm shift in NLP, where larger models not only capture more complex patterns in data but also

exhibit improved generalization across diverse applications. However, the scaling of these models imposes significant computational and memory demands, rendering them impractical for deployment on resource-constrained devices such as smartphones.

In an effort to overcome challenges, various model compression techniques have been proposed to mitigate these limitations, including quantization (Feng et al., 2024; Gupta et al., 2015; Kirtas et al., 2022), pruning (Ayinde et al., 2019; Han et al., 2015; Poyatos et al., 2023), knowledge distillation (Hinton, 2015; Qiu et al., 2024; Zhou et al., 2023), and low-rank approximation (Jaderberg et al., 2014; Pan et al., 2023; Qiu et al., 2022). These techniques aim to reduce the resource footprint of LLMs, making them more practical for applications. However, existing compression techniques often apply a uniform approach across all layers, overlooking the operational distinctions among them, which can result in varying impacts on model performance. Although these methods

\* Corresponding author.

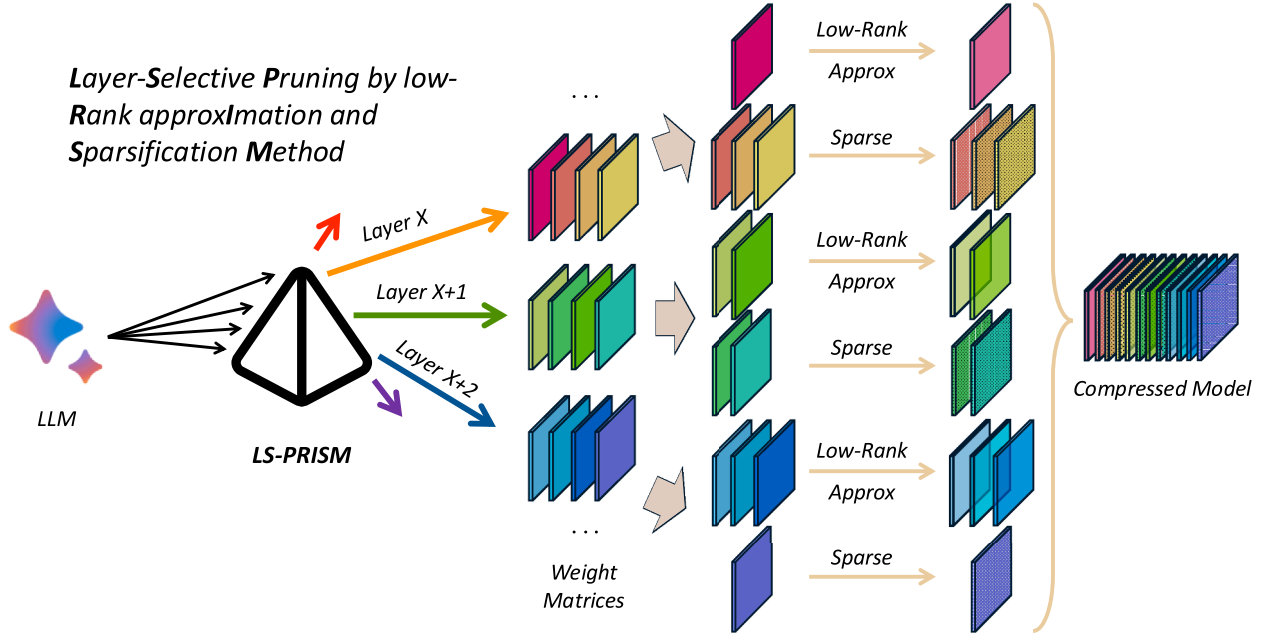
E-mail addresses: [rstaob@bjtu.edu.cn](mailto:rstaob@bjtu.edu.cn) (R. Tao), [hairong.chen.david@outlook.com](mailto:hairong.chen.david@outlook.com) (H. Chen), [yuzhe.guo@bjtu.edu.cn](mailto:yuzhe.guo@bjtu.edu.cn) (Y. Guo), [wangjk@mail.zgclab.edu.cn](mailto:wangjk@mail.zgclab.edu.cn) (J. Wang), [20230219@nuc.edu.cn](mailto:20230219@nuc.edu.cn) (B. Wang), [rmi@bjtu.edu.cn](mailto:rmi@bjtu.edu.cn) (R. Ni), [yzhao@bjtu.edu.cn](mailto:yzhao@bjtu.edu.cn) (Y. Zhao).

<https://doi.org/10.1016/j.neunet.2025.107909>

Received 29 September 2024; Received in revised form 30 May 2025; Accepted 24 July 2025

Available online 27 July 2025

0893-6080/© 2025 Elsevier Ltd. All rights reserved, including those for text and data mining, AI training, and similar technologies.



**Fig. 1.** Framework of the proposed LS-PRISM for efficient compression of Large Language Models (LLMs). LS-PRISM integrates dynamic low-rank approximation and unstructured sparsification to achieve model compression while preserving performance.

offer some alleviation, they frequently lead to significant performance degradation, particularly when aggressive compression is employed or when applied to smaller-scale LLMs. This degradation presents a substantial challenge, as it diminishes the core benefits of LLMs, particularly accuracy and efficiency.

Moreover, the lack of hardware support for compressed models and the significant computational resources required for the compression process further complicate the landscape. Many existing frameworks do not fully capitalize on the potential efficiencies offered by compression techniques, which impedes the practical deployment and reproducibility of these methods in real-world scenarios. This gap highlights the need for more sophisticated and tailored solutions that can effectively balance the trade-offs between model size, computational efficiency, and model's performance.

Despite the variety of existing methods, most compression strategies treat all layers equally, ignoring inter-layer heterogeneity and leading to suboptimal performance retention. Moreover, they often lack flexibility in adapting compression intensity to layer-specific redundancy, particularly under tight resource constraints. These limitations motivate the need for a more adaptive and performance-aware compression strategy, one that LS-PRISM is designed to fulfill. To address these challenges, in this paper, we introduce a novel method named **Layer-Selective Pruning via low-Rank Approximation and Sparsification Method (LS-PRISM)**, a novel approach designed to compress LLMs while preserving their performance. Our method employs a dynamic selection mechanism that iteratively applies low-rank approximation to specific matrices within each layer, guided by their impact on the model's accuracy and loss, with ranks determined adaptively through a Dynamic Rank Selection method. By selectively approximating matrices that positively contribute to performance, we reduce the model size while minimizing accuracy loss. Additionally, we apply unstructured pruning to the remaining original matrices to induce sparsity, followed by optional LoRA fine-tuning (Hu et al., 2021) to recover performance loss. Fig. 1 illustrates the overall framework of LS-PRISM, highlighting the integration of dynamic low-rank approximation and sparsification stages to achieve efficient model compression.

Our main contributions are as follows:

- We propose a method called LS-PRISM, which addresses the operational differences across model layers. It dynamically identifies and compresses key matrices within LLMs based on performance metrics, focusing on the most critical components to enhance model performance and reduce overall complexity adaptively.
- We combine unstructured pruning with low-rank approximation to further increase sparsity. This integration achieves higher compression ratios while maintaining model accuracy, ensuring efficient performance on resource-constrained devices.
- Through comprehensive experiments, we demonstrate that LS-PRISM achieves substantial compression with minimal degradation in performance. Additionally, we provide an analysis of the impact of calibration data size on performance, as well as the effectiveness of combining low-rank approximation with sparsification and fine-tuning.

## 2. Related work

### 2.1. Model compression techniques

**Quantization** (Qin et al., 2020) have become one of the most popular compression methods for traditional and basic models, especially LLMs. BiBERT (Qin et al., 2022), as the first transformer work with extreme 1-bit quantization, has significantly inspired subsequent work. As the first LLM 1-bit PTQ scheme, BiLLM (Huang et al., 2024a) points out a promising path to achieve efficient LLM, and DB-LLM (Chen et al., 2024) further strikes a balance between the efficiency and accuracy of binarized LLMs. IR-QLoRA (Qin et al., 2024) first introduced a new information-theoretic perspective in the quantization of LLMs and comprehensively improved the low-rank quantization fine-tuning of LLMs. Slim-LLM (Huang et al., 2024b) first theoretically pointed out the structural aggregation effect of salient weights in LLMs.

**Pruning** removes redundant or less important weights from the network to decrease its complexity. *Structured pruning* eliminates entire filters, channels, or layers, resulting in models that are more amenable to

parallel computation and easier to deploy on standard hardware (Wang et al., 2019). However, structured pruning often leads to significant performance degradation because it indiscriminately removes large blocks of parameters, which can disrupt the learned representations within the model. This loss in performance makes structured pruning less desirable for applications requiring high accuracy.

In contrast, *unstructured pruning* removes individual weights, leading to sparse weight matrices that can be compressed further (Han et al., 2015). However, while unstructured sparsity provides more flexibility, it often does not lead to actual reductions in model size or memory usage without specialized hardware or optimized libraries. Moreover, maintaining the sparsity patterns during training and fine-tuning adds complexity to the deployment pipeline.

**Knowledge Distillation** transfers knowledge from a large “teacher” model to a smaller “student” model by training the student to mimic teacher’s outputs (Hinton, 2015). This method can produce compact models that perform comparably to their larger counterparts without the need to store all the original parameters (Zhu et al., 2024). However, the distillation process is resource-intensive, requiring substantial computational power and time, which limits its accessibility to large organizations with significant infrastructure (Zhu et al., 2024). Additionally, only organizations with access to high-quality, large-scale pre-trained models can effectively utilize knowledge distillation to produce superior small models. For instance, Google’s Gemma2-2B (Team et al., 2024b) model is an example of a small model distilled from a large pre-trained teacher model, Gemma2-27B (Team et al., 2024b).

**Low-Rank Approximation** leverages the inherent redundancy in neural network weights by approximating weight matrices with lower-rank matrices (Jaderberg et al., 2014). Techniques like Singular Value Decomposition (SVD) decompose weight matrices into products of smaller matrices, thereby reducing both the number of parameters and the computational cost associated with large-scale models. Despite its effectiveness, determining the appropriate rank to balance compression and performance remains a significant challenge, as overly aggressive approximation can degrade model accuracy.

## 2.2. SVD-based low-rank approximation methods

Singular Value Decomposition (SVD) has been widely applied in model compression for its ability to approximate large neural network weight matrices by retaining only the most significant singular values (Jaderberg et al., 2014). Recent approaches, such as ASVD and SVD-LLM, have made significant advancements by addressing key limitations in traditional SVD-based methods.

ASVD introduces an activation-aware mechanism, allowing for adaptive rank selection based on the importance of activations. This improves the compression process by focusing on critical network components (Yuan et al., 2023). Similarly, SVD-LLM employs a truncation-aware and parameter-update strategy, where the compression loss is directly mapped to the truncated singular values, minimizing performance degradation (Wang et al., 2024b). Both methods achieve impressive compression ratios and help optimize memory usage.

However, despite these improvements, challenges remain. Both ASVD and SVD-LLM still involve significant computational overhead during the decomposition process, with memory usage and computation time remaining substantial. The methods apply low-rank approximations uniformly across all layers, while yielding high compression rates, can lead to noticeable performance degradation. This is particularly evident in layers where approximation errors have a more pronounced effect on overall model performance. Addressing these issues requires further refinement in the decomposition strategy to tailor the compression more effectively to the unique sensitivities of each layer, without overly increasing computation and memory demands.

## 2.3. Layer selective mechanisms for neural network compression

Sharma et al. (2023) introduced a LAYER SElective Rank reduction (LASER) technique, which demonstrated that selectively applying low-rank approximation to specific layers, particularly MLP layers, could effectively enhance model performance. Their findings also suggested that the attention layers exhibited more variability, making it challenging to identify consistent trends. Building upon this insightful work, we adopt a more comprehensive approach by applying low-rank approximation across both MLP and attention layers. Additionally, while their work focused on understanding layer-wise performance, our approach integrates this idea with a broader goal of model compression, combining low-rank approximation with sparsification and LoRA fine-tuning. This allows us to achieve significant compression while maintaining or improving model performance, offering a scalable and effective solution for resource-constrained environments.

## 3. Methodology

### 3.1. Overview of the proposed method

Our proposed method, **LS-PRISM** (Layer-Selective Pruning via low-Rank approximation and Sparsification Method), compresses Large Language Models (LLMs) through a two-stage process designed to maximize compression efficiency while preserving or enhancing model performance.

The two stages are as follows:

**Stage 1: Dynamic Low-Rank Approximation** involves iteratively applying low-rank approximations to carefully selected weight matrices within each model layer. The selection and approximation processes are guided by their direct impact on key performance metrics, specifically accuracy and loss, ensuring that only matrices whose approximations maintain or enhance model performance are compressed. To achieve this effectively, we employ a dynamic rank selection method combining KMeans clustering and Isolation Forest anomaly detection (KMIF). This adaptive strategy identifies the optimal truncation rank individually for each matrix based on its singular value distribution, thus preserving critical structural information while significantly reducing the number of parameters.

**Stage 2: Sparsification and Optional Fine-Tuning with LoRA** entails applying unstructured pruning using the Wanda method to the remaining original weight matrices. This induces sparsity and further reduces the model size. To mitigate any potential performance degradation resulting from pruning, we optionally fine-tune the pruned layers using LoRA (Low-Rank Adaptation), which introduces minimal additional parameters to recover lost performance without significantly altering the model’s sparsity.

This two-stage approach leverages the complementary strengths of low-rank approximation and sparsification, enhanced by efficient fine-tuning, to achieve substantial model compression with minimal impact on performance.

### 3.2. Dynamic low-rank approximation (stage 1)

**Establishing performance baseline.** We commence by evaluating the original model, denoted as  $M_{orig}$ , on a calibration dataset  $\mathcal{D}_{calib}$ . This evaluation provides baseline-accuracy ( $ACC_{orig}$ ) and loss ( $\mathcal{L}_{orig}$ )-which serve as essential reference points for assessing the effectiveness of subsequent compression procedures.

**Dynamic rank selection.** Choosing an appropriate truncation rank is essential to effectively compress the model while maintaining performance. For this purpose, we adopt a dynamic rank selection strategy based on combining KMeans clustering and Isolation Forest anomaly detection (KMIF) - an approach frequently employed in image compression tasks due to its effectiveness in capturing significant structural

**Algorithm 1** Dynamic rank selection.

---

**Require:** Weight Matrix  $\mathbf{W}$ , Singular Values  $\{\sigma_i\}_{i=1}^p$   
**Ensure:** Selected Rank  $r$

- 1: Perform SVD:  $\mathbf{W} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$
- 2: Apply log-transform to singular values:  $\hat{\sigma}_i = \log(\sigma_i)$
- 3: Cluster singular values using KMeans into two clusters,  $C_{dom}$  and  $C_{sub}$
- 4: Identify the dominant cluster  $C_{dom}$  containing the largest singular value
- 5: Apply Isolation Forest anomaly detection to singular values in  $C_{dom}$
- 6: Find the first significant discontinuity in the anomaly sequence to determine  $r$
- 7: **return**  $r$

---

information from singular value distributions (Falini, 2022). The detailed steps of Dynamic Rank Selection method are outlined in Algorithm 1.

Specifically, for a given weight matrix  $\mathbf{W} \in \mathbb{R}^{m \times n}$ , we first perform Singular Value Decomposition (SVD) to decompose it into orthogonal matrices and singular values:

$$\mathbf{W} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$$

where  $\mathbf{\Sigma} = \text{diag}(\sigma_1, \sigma_2, \dots, \sigma_p)$ ,  $p = \min(m, n)$ , and singular values  $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_p$  are ordered in descending magnitude. Given the typically wide range of singular value magnitudes, we apply a logarithmic transformation to stabilize the variance and improve interpretability:

$$\hat{\sigma}_i = \log(\sigma_i), \quad i = 1, 2, \dots, p.$$

Next, we employ KMeans clustering to partition the transformed singular values  $\{\hat{\sigma}_i\}_{i=1}^p$  into two distinct clusters, defined as:

$$C_1, C_2 = \text{KMeans}(\{\hat{\sigma}_i\}_{i=1}^p, k=2).$$

Typically, one cluster ( $C_{dom}$ ) includes singular values associated with critical information, while the other cluster ( $C_{sub}$ ) contains singular values with relatively minimal impact. We identify the dominant cluster  $C_{dom}$  by selecting the cluster that includes the largest singular value:

$$C_{dom} = \begin{cases} C_1 & \text{if } \max_{\hat{\sigma} \in C_1} \hat{\sigma} > \max_{\hat{\sigma} \in C_2} \hat{\sigma}, \\ C_2 & \text{otherwise.} \end{cases}$$

To further pinpoint significant singular values within the dominant cluster, we leverage Isolation Forest, an anomaly detection technique effective at identifying outliers in data distributions. Isolation Forest assigns an anomaly label to each singular value in  $C_{dom}$ :

$$\text{IForest}(\hat{\sigma}) = \begin{cases} -1 & \text{if } \hat{\sigma} \text{ is an outlier,} \\ +1 & \text{otherwise.} \end{cases}$$

Let  $O = \{i \mid \text{IForest}(\hat{\sigma}_i) = -1, \hat{\sigma}_i \in C_{dom}\}$  denote the ordered set of outlier indices. We determine the optimal truncation rank  $r$  by identifying the first significant discontinuity (gap greater than one) in the ordered sequence of indices in  $O$ :

$$r = \min \{O_k : O_{k+1} - O_k > 1\}.$$

If no discontinuity is found (i.e., the set  $O$  is continuous), we conservatively retain all singular values as significant by setting  $r = p$ . With this dynamically selected rank  $r$ , we approximate the weight matrix  $\mathbf{W}$  by retaining only the top- $r$  singular vectors and singular values, yielding:

$$\mathbf{W}' = \mathbf{U}_r \mathbf{\Sigma}_r \mathbf{V}_r^\top,$$

where  $\mathbf{U}_r \in \mathbb{R}^{m \times r}$ ,  $\mathbf{\Sigma}_r = \text{diag}(\sigma_1, \dots, \sigma_r) \in \mathbb{R}^{r \times r}$ , and  $\mathbf{V}_r \in \mathbb{R}^{n \times r}$ .

This approach ensures that the selected rank adapts precisely to each matrix's intrinsic structure, preserving critical information while significantly reducing the parameter count. Consequently, our dynamic rank

**Algorithm 2** Dynamic layer selection for low-rank approximation.

---

**Require:** Original model  $M_{orig}$ , calibration dataset  $D_{calib}$ , baseline accuracy  $ACC_{orig}$  and loss  $\mathcal{L}_{orig}$ , candidate layers set  $\mathcal{L}$ .  
**Ensure:** Compressed model  $M_{LR}$  with selected layers approximated.

- 1: Initialize  $M_{LR} \leftarrow M_{orig}$
- 2: **for** each layer  $\ell \in \mathcal{L}$  **do**
- 3:   Obtain the weight matrix  $\mathbf{W}_\ell$  of layer  $\ell$
- 4:   Compute dynamically selected rank  $r_\ell$  using Algorithm 1 on  $\mathbf{W}_\ell$
- 5:   Approximate  $\mathbf{W}_\ell$  by its low-rank form  $\mathbf{W}'_\ell = \mathbf{U}_{r_\ell} \mathbf{\Sigma}_{r_\ell} \mathbf{V}_{r_\ell}^\top$
- 6:   Temporarily replace  $\mathbf{W}_\ell$  in  $M_{LR}$  with  $\mathbf{W}'_\ell$
- 7:   Evaluate the updated model  $M'_{LR}$  on dataset  $D_{calib}$ , obtaining accuracy  $ACC'$  and loss  $\mathcal{L}'$
- 8:   **if**  $(ACC' \geq ACC_{orig})$  and  $(\mathcal{L}' \leq \mathcal{L}_{orig})$  **then**
- 9:     Permanently accept the approximation for layer  $\ell$
- 10:   **else**
- 11:     Restore the original matrix  $\mathbf{W}_\ell$  in  $M_{LR}$
- 12:   **end if**
- 13: **end for**
- 14: **return**  $M_{LR}$

---

selection strategy effectively balances model compression and predictive accuracy.

*Iterative matrix selection and low-rank approximation.* Applying the KMIF-based dynamic rank selection, we proceed iteratively through each critical weight matrix within the model. For example, in the Gemma-2B model, we focus primarily on essential components such as query ( $\mathbf{W}_Q$ ), key ( $\mathbf{W}_K$ ), value ( $\mathbf{W}_V$ ), and output ( $\mathbf{W}_O$ ) matrices within the attention mechanism, as well as the  $\mathbf{W}_{gate}$ ,  $\mathbf{W}_{up}$ , and  $\mathbf{W}_{down}$  matrices in MLP layers.

Each candidate weight matrix  $\mathbf{W}$  undergoes the described dynamic low-rank approximation process, resulting in an approximated matrix  $\mathbf{W}'$ . Subsequently, we replace the original matrix  $\mathbf{W}$  with its approximation  $\mathbf{W}'$  and re-evaluate the modified model  $M'$  on the calibration dataset  $D_{calib}$ , obtaining updated performance metrics  $ACC'$  and  $\mathcal{L}'$ . If the approximated model's accuracy and loss meet the criteria ( $ACC' \geq ACC_{orig}$  and  $\mathcal{L}' \leq \mathcal{L}_{orig}$ ), the approximated matrix is retained; otherwise, the original matrix is restored. This iterative evaluation ensures only beneficial approximations are preserved, thereby safeguarding the overall effectiveness and robustness of the compressed model.

Fig. 2 provides a comprehensive illustration of this iterative and dynamic process.

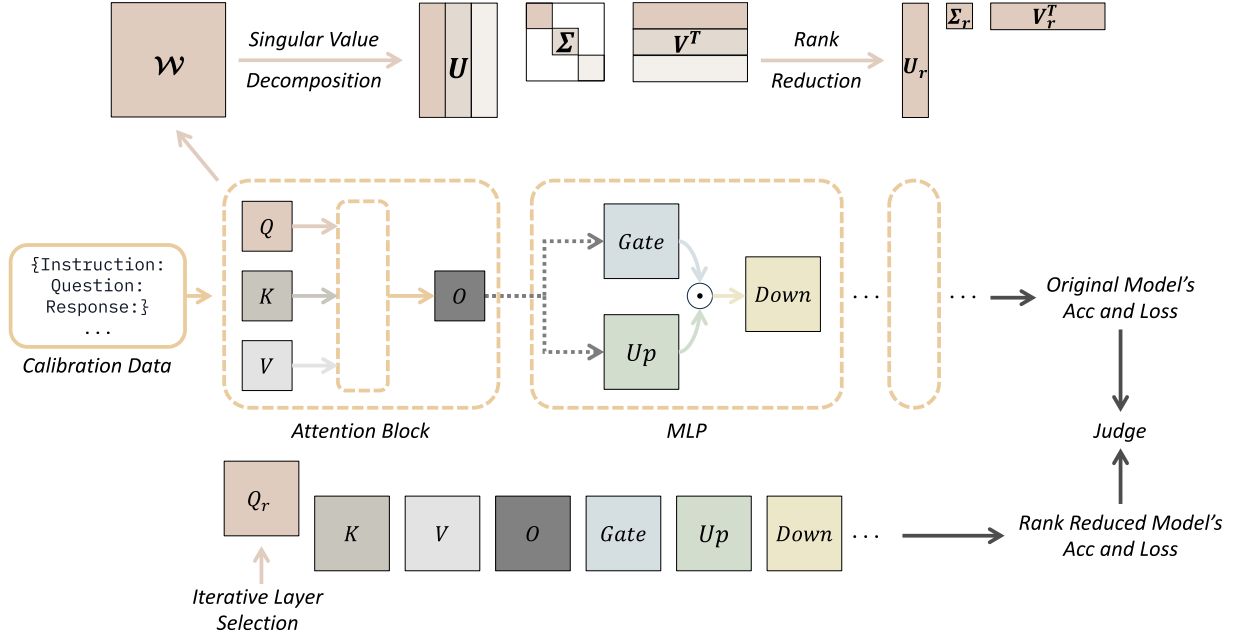
The iterative process for automatically selecting layers for low-rank approximation based on their impact on model performance is summarized in Algorithm 2.

*Model update.* Upon completion of the iterative procedure for all targeted matrices, we obtain a partially compressed model, denoted as  $M_{LR}$ . In  $M_{LR}$ , selected matrices have been successfully replaced with their low-rank approximations, while others remain unchanged to maintain model integrity. For storage efficiency, the approximated matrices are preserved in their decomposed form ( $\mathbf{U}_r \mathbf{\Sigma}_r \mathbf{V}_r^\top$ ), significantly reducing model size while retaining essential performance characteristics.

### 3.3. Sparsification and optional fine-tuning with LoRA (stage 2)

*Sparsification.* To further compress the model  $M_{LR}$ , we apply unstructured pruning using the Wanda method to the remaining original weight matrices that were not selected for low-rank approximation in Stage 1. Wanda leverages calibration data to evaluate the importance of each weight based on activation norms, effectively identifying and pruning the least critical weights:

$$s_{ij} = |\mathbf{W}_{ij}| \cdot \|\mathbf{X}_j\|_2$$



**Fig. 2.** Overview of Stage 1: Dynamic Low-Rank Approximation. This stage involves iteratively applying SVD-based low-rank approximations to selected weight matrices based on their impact on model performance.

where  $s_{ij}$  is the pruning score for weight  $W_{ij}$ , and  $\|X_j\|_2$  is the  $\ell_2$  norm of the  $j$ -th input activation vector. We prune a fraction  $s$  of the weights with the lowest scores, inducing sparsity in the weight matrices.

**Optional fine-tuning with LoRA.** To counteract any potential performance loss resulting from pruning, we optionally fine-tune the pruned layers using LoRA (Low-Rank Adaptation). LoRA introduces additional low-rank matrices to facilitate efficient fine-tuning without significantly increasing the parameter count:

$$W_{\text{tuned}} = W_{\text{sparse}} + \Delta W, \quad \Delta W = AB^T$$

where  $A \in \mathbb{R}^{d \times r}$  and  $B \in \mathbb{R}^{r \times d}$  are low-rank adaptation matrices of rank  $r$ . During fine-tuning, only  $A$  and  $B$  are updated, while  $W_{\text{sparse}}$  remains fixed.

LoRA offers several advantages over full fine-tuning, including parameter efficiency, maintained sparsity, reduced computational overhead, and flexibility in adapting to various tasks. By introducing only a minimal number of additional parameters, LoRA ensures that the sparsity level of the compressed model remains largely unchanged, preserving the efficiency gains from both low-rank approximation and unstructured pruning. Furthermore, the low computational overhead of LoRA fine-tuning makes it feasible to fine-tune large and sparsified models on resource-constrained hardware. Fig. 3 provides a detailed overview of Stage 2, highlighting the application of Wanda sparsification along with the optional integration of LoRA for fine-tuning.

### 3.4. Final model construction

The final compressed model, denoted as  $M_{\text{compressed}}$ , integrates the outcomes of both compression stages:

$$M_{\text{compressed}} = \{W_{LR}, W_{\text{sparse}}, \Delta W\}$$

where the  $W_{LR}$  represents the low-rank approximated matrices from Stage 1, the  $W_{\text{sparse}}$  denotes the sparsified matrices obtained through Wanda pruning, and  $\Delta W$  is the optional low-rank adaptation matrices introduced by LoRA.

This comprehensive approach combines the strengths of low-rank approximation, unstructured sparsification, and efficient fine-tuning

to achieve significant model compression with minimal performance degradation. By dynamically selecting matrices for low-rank approximation based on their impact on performance and strategically applying sparsification, LS-PRISM ensures that the compressed model remains both efficient and effective, suitable for deployment in resource-constrained environments.

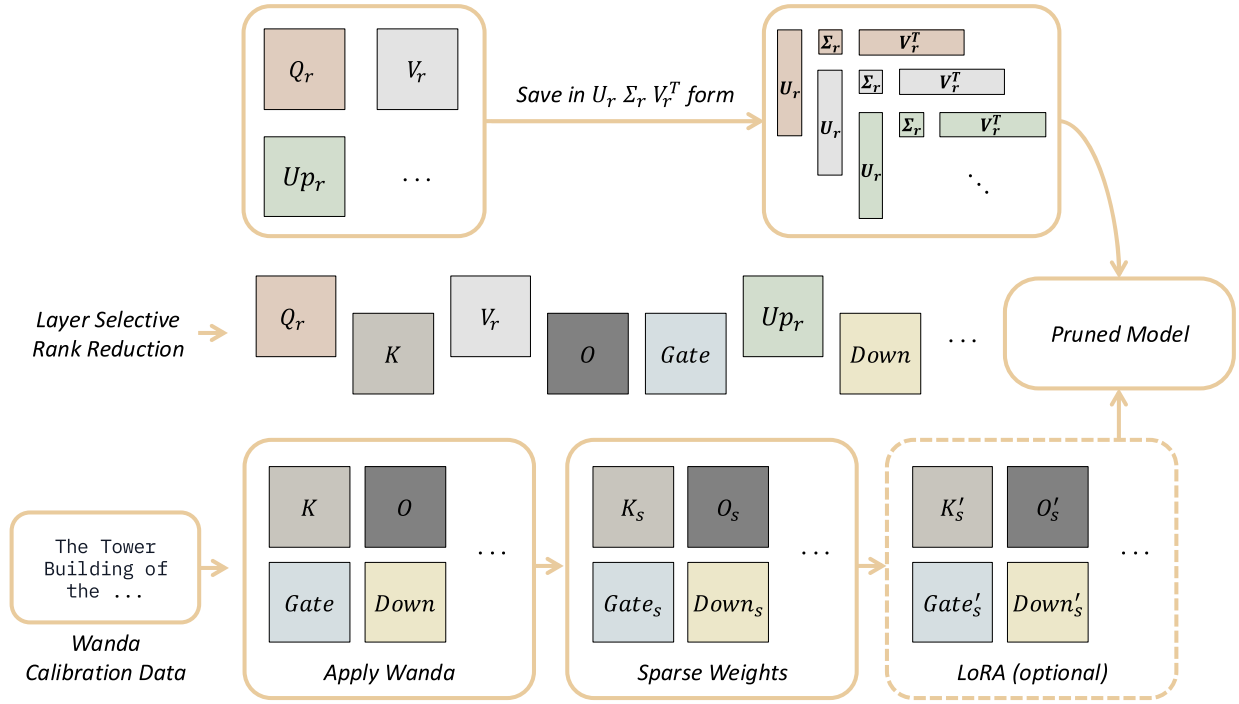
## 4. Experiments

### 4.1. Experimental setup

To evaluate the effectiveness of our proposed method, **LS-PRISM**, we conducted a series of experiments on zero-shot natural language understanding tasks using subsets of standard NLP datasets (Liu et al., 2024a; Xian et al., 2018). As baselines, we selected three models: **Google's** Gemma-2B (Team et al., 2024a), a 2.5 billion-parameter model, **Microsoft's** Phi-1.5 (Li et al., 2023a), a 1.4 billion-parameter model, and **Meta's** LLaMA3.2-1B (Grattafiori et al., 2024), a 1.2 billion-parameter model. These models, while all belonging to the small-scale category, differ in architecture and training strategies, allowing us to analyze LS-PRISM's effectiveness across different model designs. Given our computational constraints, we focus on models with fewer than 3 billion parameters, ensuring feasibility while still capturing meaningful performance variations.

**Calibration data.** For the calibration dataset  $D_{\text{calib}}$ , we utilized two distinct sources tailored to each stage of LS-PRISM. Specifically, we employed the CounterFact dataset (Meng et al., 2022) for the Dynamic Low-Rank Approximation stage and the Wikitext2 dataset (Merity et al., 2016) for the Wanda sparsification stage. To assess the impact of  $D_{\text{calib}}$  data size on Stage 1, we experimented with varying numbers of samples from the CounterFact dataset.

**Evaluation tasks.** The compressed models were evaluated on zero-shot settings for the BoolQ (Clark et al., 2019) and Recognizing Textual Entailment (RTE) (Bentivogli et al., 2009) tasks from the GLUE benchmark (Wang, 2018), as well as the ARC-Easy and ARC-Challenge



**Fig. 3.** Overview of Stage 2: Sparsification and Optional Fine-Tuning with LoRA. This stage applies unstructured pruning to the remaining weight matrices and optionally fine-tunes the model using LoRA to recover any performance loss.

(Clark et al., 2018), HellaSwag (Zellers et al., 2019), Winogrande (Sakaguchi et al., 2019), and OpenBookQA (Mihaylov et al., 2018) benchmarks. BoolQ evaluates a model's ability to answer binary questions by determining whether a passage supports a "yes" or "no" answer. RTE assesses the model's capacity for textual entailment, requiring it to decide if a hypothesis logically follows from a given premise. ARC-Easy and ARC-Challenge test a model's ability to answer multiple-choice science questions with increasing difficulty. HellaSwag measures common-sense reasoning through sentence completion. Winogrande is designed to evaluate pronoun resolution in challenging coreference cases, while OpenBookQA assesses open-domain knowledge and reasoning capabilities.

**Compression configurations.** We explored different configurations of LS-PRISM to understand the contributions of each component:

1. Applying only Stage 1: Dynamic Low-Rank Approximation.
2. Combining Stage 1 with Wanda sparsification.
3. Incorporating LoRA fine-tuning after sparsification.

**Implementation details.** The rank  $r$  for low-rank approximation was determined based on preliminary experiments to achieve a balance between compression and performance. For Wanda pruning, we set the target sparsity level to 50%. This choice was guided by previous studies such as Wanda (Sun et al., 2023) and SparseGPT (Frantar & Alistarh, 2023), which demonstrated that sparsity levels around 40–50% provide an effective balance between compression efficiency and performance retention. Specifically, higher sparsity levels (above 70%) typically incur significant accuracy degradation, whereas lower sparsity levels (below 30%) offer limited practical benefits in parameter reduction (Frantar & Alistarh, 2023; Sun et al., 2023). Additionally, a sparsity level of 50% aligns well with existing sparse matrix libraries, ensuring compatibility with hardware-accelerated inference. In configurations involving LoRA fine-tuning, the rank of the adaptation matrices was fixed at  $r = 4$ ,

and the LoRA fine-tuning was performed using only 300 samples of the dataset. All experiments were conducted on servers equipped with 2 NVIDIA RTX 4090 GPUs.

## 4.2. Results

### 4.2.1. Impact of calibration data size on stage 1

The first set of experiments aimed to determine the optimal size of the calibration dataset for the Dynamic Low-Rank Approximation stage on the Gemma-2B model. Table 1 presents the results of varying the number of CounterFact data samples used during this stage.

As shown in Table 1, smaller calibration data sizes, such as 5 samples, led to significant reductions in both parameter count and model size, but this aggressive compression caused a considerable drop in accuracy, decreasing to 43.24% from the original 62.94%. When the calibration data size was increased to 10 samples, accuracy improved to 56.91%, and with 20 samples, the model achieved an accuracy of 59.82%. This data size also resulted in a notable 11% reduction in both parameter count and model size, representing a strong balance between compression and performance, highlighted in green.

Increasing the calibration data to 30 samples did not lead to further accuracy gains, and performance slightly decreased to 59.30%. At the largest calibration data size of 100 samples, the model nearly matched its original performance with an accuracy of 61.35%, but the process took considerably longer, with the compression phase requiring 39.4 min. While using 100 samples yielded the best performance, the longer time required may not justify the marginal improvement compared to using fewer samples, which provides a more efficient trade-off between compression, accuracy, and time.

These results indicate that an optimal calibration data size exists, with 20 to 30 samples being effective in achieving substantial compression without a significant loss in accuracy or excessive computational overhead.

**Table 1**

Impact of Different Calibration Data Sizes on Stage 1 Zero-shot Results of Gemma-2B. The results highlighted in green represent the best trade-offs between compression and performance. “↓” indicates the reduction from the original model.

| Experiment      | Accuracy (%) | Parameters (B)        | Size (GB)           | Time (min) |
|-----------------|--------------|-----------------------|---------------------|------------|
| Gemma-2B        | 62.94        | 2.506                 | 4.67                | -          |
| Data size = 5   | 43.24        | 2.013(↓ 0.493)        | 3.75(↓ 0.92)        | 3.2        |
| Data size = 10  | 56.91        | 2.293(↓ 0.213)        | 4.27(↓ 0.40)        | 5.1        |
| Data size = 20  | <b>59.82</b> | <b>2.229(↓ 0.277)</b> | <b>4.15(↓ 0.52)</b> | <b>9.0</b> |
| Data size = 30  | 59.30        | 2.312(↓ 0.194)        | 4.31(↓ 0.36)        | 12.7       |
| Data size = 50  | 44.28        | 2.254(↓ 0.252)        | 4.20(↓ 0.47)        | 20.3       |
| Data size = 100 | <b>61.35</b> | <b>2.201(↓ 0.305)</b> | <b>4.10(↓ 0.57)</b> | 39.4       |

Note: Parameters are in billions (B), and Size is in gigabytes (GB).

**Table 2**

Impact of Adding Wanda and LoRA on Zero-shot Results for Gemma-2B. The results highlighted in green represent the best trade-offs between compression, sparsity and performance. “↓” indicates the reduction from the original model.

| Experiment        | Accuracy (%) |              | Parameters (B) | Size (GB)    | Sparsity      | Time (min) |
|-------------------|--------------|--------------|----------------|--------------|---------------|------------|
|                   | BoolQ        | RTE          |                |              |               |            |
| Gemma-2B          | 62.94        | 51.99        | 2.506          | 4.67         | 0.0000        | -          |
| Wanda[54]         | 50.61        | 49.10        | 2.506( - )     | 4.67( - )    | <b>0.5000</b> | 1.6        |
| Wanda + LoRA[26]  | 52.48        | 50.54        | 2.506( - )     | 4.67( - )    | 0.4976        | 2.8        |
| Stage 1           | 59.82        | 51.99        | 2.229(↓ 0.277) | 4.15(↓ 0.52) | 0.0000        | 9.0        |
| Stage 1 + LoRA    | <b>61.59</b> | 51.26        | 2.229(↓ 0.277) | 4.15(↓ 0.52) | 0.0000        | 12.5       |
| LS-PRISM w/o LoRA | 60.70        | <b>53.06</b> | 2.229(↓ 0.277) | 4.15(↓ 0.52) | 0.4884        | 11.6       |
| LS-PRISM(ours)    | <b>61.56</b> | <b>52.71</b> | 2.229(↓ 0.277) | 4.15(↓ 0.52) | <b>0.4494</b> | 13.3       |

Note: Parameters are in billions (B), and Size is in gigabytes (GB), w/o denotes without.

#### 4.2.2. Effectiveness of combining sparsification and LoRA

The second set of experiments evaluated the combined effect of applying Wanda sparsification and LoRA fine-tuning after Dynamic Low-Rank Approximation on the Gemma-2B model. Table 2 summarizes the impact of these additional compression steps on the model’s performance.

Applying Wanda alone resulted in a moderate decrease in accuracy, with BoolQ dropping to 50.61 % and RTE to 49.10%, while achieving a sparsity of 50 %. Introducing LoRA fine-tuning after Wanda slightly improved the performance to 52.48 % on BoolQ and 50.54 % on RTE, demonstrating that LoRA can recover some of the performance lost due to sparsification.

When Dynamic Low-Rank Approximation (Stage 1) was applied alone, the model retained most of its original performance, achieving 59.82 % on BoolQ with a 12 % reduction in parameters. Adding LoRA fine-tuning to Stage 1 further enhanced the performance to 61.59 % on BoolQ, nearing the original accuracy. Combining Stage 1 with Wanda sparsification (LS-PRISM without LoRA) resulted in significant sparsity and improved performance on RTE to 53.06 %, while maintaining a high BoolQ accuracy.

Finally, the complete LS-PRISM method, which integrates Dynamic Low-Rank Approximation, Sparsification, and LoRA fine-tuning, achieved the best overall performance. It maintained a high BoolQ accuracy of 61.56 % and an improved RTE score of 52.71 %, while reducing the model parameters by 0.277 billion (11 %), while inducing a sparsity of approximately 45 %.

These findings demonstrate that combining Wanda sparsification with Dynamic Low-Rank Approximation and optionally fine-tuning with

LoRA can effectively compress the model while preserving its performance.

#### 4.2.3. Compression performance on smaller-scale model

To evaluate the scalability of LS-PRISM, we applied our compression method to the smaller Phi-1.5 model, which has lower parameter redundancy compared to larger models like Gemma-2B. The effects of compression and performance retention differ significantly due to the reduced overparameterization in smaller models. As illustrated in Table 3, the impact of calibration data size on the compression performance of the Phi-1.5 model is significant. When utilizing a smaller calibration dataset size of 30 samples, LS-PRISM achieves a substantial reduction in both parameters and model size (31 %). However, this aggressive compression leads to a pronounced decline in performance, particularly on the RTE task, where accuracy drops from 58.48 % to 48.38 %. The BoolQ task also experiences a decrease in accuracy. The poorer performance on RTE can be attributed to its inherent complexity and the model’s reduced ability to capture nuanced textual entailment relationships with limited calibration data. Smaller calibration datasets may fail to provide sufficient diversity and representativeness, resulting in less effective low-rank approximations and pruning decisions that disproportionately impact more intricate tasks like RTE. Conversely, increasing the calibration dataset size to 100 samples mitigates this issue, restoring RTE accuracy to 62.09 % while maintaining a comparable reduction in parameters and model size (22 %). This improvement underscores the necessity of ample calibration data to preserve the model’s capacity to perform complex tasks post-compression. Overall, the results highlight the critical role of calibration dataset size in balancing compression

**Table 3**

Impact of Different Calibration Data Sizes on Stage 1 Zero-shot Results of Phi-1.5. The results highlighted in green represent the best trade-offs between compression and performance. “↓” indicates the reduction from the original model.

| Experiment   | Accuracy (%) |              | Parameters (B) | Size (GB)    | Sparsity | Time (min) |
|--------------|--------------|--------------|----------------|--------------|----------|------------|
|              | BoolQ        | RTE          |                |              |          |            |
| Phi-1.5      | 75.02        | 58.48        | 1.418          | 5.28         | 0.0000   | -          |
| Datasize=30  | <b>62.02</b> | 48.38        | 0.978(↓ 0.440) | 3.64(↓ 1.64) | 0.0000   | 12.8       |
| Datasize=100 | 61.13        | <b>62.09</b> | 1.108(↓ 0.310) | 4.12(↓ 1.16) | 0.0000   | 60.0       |

Note: Parameters are in billions (B), and Size is in gigabytes (GB).

**Table 4**

Impact of Adding Wanda and LoRA on Zero-shot Results for Phi-1.5. The datasize is set to 30 for efficiency. Results in green represent the best balance between compression and performance.

| Experiment        | Accuracy (%) |              | Parameters (B) | Size (GB)    | Sparsity      | Time (min) |
|-------------------|--------------|--------------|----------------|--------------|---------------|------------|
|                   | BoolQ        | RTE          |                |              |               |            |
| Phi-1.5           | 75.02        | 58.48        | 1.418          | 5.28         | 0.0000        | -          |
| Wanda             | 71.01        | 59.20        | 1.418( - )     | 5.28( - )    | <b>0.5000</b> | 0.5        |
| Wanda + LoRA      | <b>71.32</b> | <b>59.96</b> | 1.418( - )     | 5.28( - )    | 0.4848        | 1.6        |
| Stage 1           | 62.02        | 48.38        | 0.978(↓ 0.440) | 3.64(↓ 1.64) | 0.0000        | 12.8       |
| LS-PRISM w/o LoRA | <b>62.14</b> | 46.57        | 0.978(↓ 0.440) | 3.64(↓ 1.64) | <b>0.4451</b> | 15.6       |
| LS-PRISM          | 61.62        | <b>50.54</b> | 0.978(↓ 0.440) | 3.64(↓ 1.64) | 0.4367        | 16.0       |

Note: Parameters are in billions (B), and Size is in gigabytes (GB). LoRA rank is set to 4. “↓” indicates the reduction from the original model. w/o denotes without.

efficiency with performance retention, especially for tasks that demand a deeper semantic understanding.

To further evaluate the impact of Wanda sparsification and LoRA fine-tuning on this smaller model, Table 4 presents the results of combining these methods with LS-PRISM. As demonstrated in Table 4, the application of LS-PRISM to the Phi-1.5 model reveals nuanced interactions between low-rank approximation, sparsification, and fine-tuning techniques. Initially, applying Wanda sparsification alone results in a moderate decrease in BoolQ accuracy from 75.02 % to 71.01 %, while slightly improving RTE accuracy from 58.48 % to 59.20 %. This improvement in RTE suggests that Wanda effectively prunes less critical weights without disrupting the model’s ability to perform complex entailment tasks. Applying Stage 1, which involves dynamic low-rank approximation, results in a significant reduction in parameters and model size (31 %). However, this aggressive compression leads to a notable decline in BoolQ accuracy to 62.02 % and a more pronounced decrease in RTE accuracy to 48.38 %. The substantial performance loss, particularly in RTE, can be attributed to the lower parameter redundancy in smaller models like Phi-1.5, which limits the extent to which low-rank approximations can be applied without compromising the model’s ability to capture intricate linguistic patterns essential for entailment tasks. Interestingly, combining Stage 1 with Wanda sparsification without LoRA fine-tuning (LS-PRISM w/o LoRA) partially mitigates the performance loss, achieving a BoolQ accuracy of 62.14 % and an RTE accuracy of 46.57 %, while further inducing sparsity (44.51 %) and maintaining the parameter and size reductions achieved by Stage 1. This suggests that the subsequent application of Wanda after low-rank approximation continues to refine the model by removing additional redundant weights, thereby enhancing efficiency without exacerbating performance degradation. Finally, the full LS-PRISM method, which integrates dynamic low-rank approximation, Wanda sparsification, and LoRA fine-tuning, achieves a balanced trade-off with BoolQ and RTE

accuracies of 61.62 % and 50.54 %, respectively. The inclusion of LoRA fine-tuning helps recover some of the performance lost during both low-rank approximation and sparsification, resulting in a more resilient compressed model.

Overall, the results highlight the delicate balance between compression and performance in smaller models with lower parameter redundancy. While low-rank approximation alone can lead to substantial efficiency gains, it may do so at the cost of significant performance losses, especially in complex tasks like RTE. Incorporating sparsification provides an additional layer of compression that, when combined with fine-tuning via LoRA, offers a more effective means of preserving model performance. This underscores the importance of a versatile compression strategy in resource-constrained environments, particularly for smaller LLMs where parameter redundancy is limited.

#### 4.2.4. Impact of singular value removal rate in stage 1: dynamic low-rank approximation

Table 5 presents the effects of varying the Singular Value Removal Rate during Stage 1: Dynamic Low-Rank Approximation on the Gemma-2B model. At lower removal rates of 10 % and 20 %, there is an initial increase in both the number of parameters and the model size, attributed to the overhead associated with storing the decomposed matrices ( $U, \Sigma, V^T$ ). Specifically, a 10 % removal rate results in a parameter increase of 5.3 % increase of model size, while a 20 % removal rate leads to a minor parameter increase of 1.3 %.

As the removal rate progresses to 30 %, the model begins to achieve actual compression, reducing parameters by 2.2 %, accompanied by a significant improvement in RTE accuracy to 59.21 %. This indicates that beyond a certain threshold, low-rank approximation effectively reduces redundancy without substantially harming performance. Increasing the removal rate to 40 % and 50 % continues this trend, with parameter

**Table 5**

Effect of Singular Value Removal Rate in Stage 1: Dynamic Low-Rank Approximation on the Gemma-2B Model. Results highlighted in green indicate the optimal balance between compression and performance. Green “↓” arrows represent reductions from the original model, while red “↑” arrows denote increases from the original model.

| Experiment           | Accuracy (%) |              | Parameters (B)         | Size (GB)            |
|----------------------|--------------|--------------|------------------------|----------------------|
|                      | BoolQ        | RTE          |                        |                      |
| 0% Removal           | 62.94        | 51.99        | 2.506                  | 4.67                 |
| 0%+Sparsity          | 50.61        | 49.10        | 2.506                  | 4.67                 |
| 10% Removal          | 62.02        | 52.35        | 2.638(↑ 0.132)         | 4.91(↑ 0.24)         |
| 10%+Sparsity         | 61.22        | 52.71        | 2.638(↑ 0.132)         | 4.91(↑ 0.24)         |
| 20% Removal          | 62.32        | 53.07        | 2.539(↑ 0.033)         | 4.73(↑ 0.06)         |
| 20%+Sparsity         | 52.97        | 49.10        | 2.539(↑ 0.033)         | 4.73(↑ 0.06)         |
| 30% Removal          | 45.78        | <b>59.21</b> | 2.451(↓ 0.055)         | 4.57(↓ 0.10)         |
| 30%+Sparsity         | 42.62        | 51.62        | 2.451(↓ 0.055)         | 4.57(↓ 0.10)         |
| 40% Removal          | 60.92        | 55.60        | 2.409(↓ 0.097)         | 4.49(↓ 0.18)         |
| 40%+Sparsity         | 57.74        | 49.82        | 2.409(↓ 0.097)         | 4.49(↓ 0.18)         |
| 50% Removal          | 60.55        | 51.99        | 2.284(↓ 0.222)         | 4.25(↓ 0.42)         |
| 50%+Sparsity         | 60.73        | 53.07        | 2.284(↓ 0.222)         | 4.25(↓ 0.42)         |
| 60% Removal          | 53.94        | 53.80        | 2.252(↓ 0.254)         | 4.19(↓ 0.48)         |
| 60%+Sparsity         | 42.17        | <b>53.43</b> | 2.252(↓ 0.254)         | 4.19(↓ 0.48)         |
| 70% Removal          | 46.33        | 53.07        | 2.298(↓ 0.208)         | 4.28(↓ 0.39)         |
| 70%+Sparsity         | 47.00        | 49.81        | 2.298(↓ 0.208)         | 4.28(↓ 0.39)         |
| 80% Removal          | 57.52        | 54.51        | 2.282(↓ 0.224)         | 4.25(↓ 0.42)         |
| 80%+Sparsity         | 54.34        | 51.26        | 2.282(↓ 0.224)         | 4.25(↓ 0.42)         |
| 90% Removal          | 59.82        | 51.99        | <b>2.229</b> (↓ 0.277) | <b>4.15</b> (↓ 0.52) |
| 90%+Sparsity         | <b>60.70</b> | <b>53.06</b> | <b>2.229</b> (↓ 0.277) | <b>4.15</b> (↓ 0.52) |
| 95% Removal          | 54.86        | 49.46        | 2.252(↓ 0.254)         | 4.20(↓ 0.47)         |
| 95%+Sparsity         | 41.40        | 45.13        | 2.252(↓ 0.254)         | 4.20(↓ 0.47)         |
| 99% Removal          | 58.47        | 49.46        | 2.315(↓ 0.191)         | 4.31(↓ 0.36)         |
| 99%+Sparsity         | 50.15        | 49.46        | 2.315(↓ 0.191)         | 4.31(↓ 0.36)         |
| <b>KMIF</b>          | <b>62.29</b> | 56.31        | 2.355(↓ 0.151)         | 4.39(↓ 0.28)         |
| <b>KMIF+Sparsity</b> | <b>62.11</b> | 53.06        | 2.355(↓ 0.151)         | 4.39(↓ 0.28)         |

Note: Parameters are in billions (B), and Size is in gigabytes (GB).

reductions of 3.9 % and 8.9 % respectively, while maintaining or slightly improving BoolQ and RTE accuracies. Notably, at a 90 % removal rate, the model achieves the highest BoolQ accuracy of 60.70 % and an RTE score of 53.06 %, alongside an 11 % reduction in parameters and model size. This suggests that higher singular value removal rates can leverage the model’s inherent parameter redundancy to achieve substantial compression without detrimental effects on performance.

However, excessively high removal rates of 95 % and 99 % begin to reverse these benefits, resulting in parameter reductions of 22.4 % and 19.1 % respectively, but with corresponding declines in accuracy, particularly evident in the RTE task. This degradation underscores the importance of balancing the degree of compression with the preservation of critical model functionalities.

The integration of Wanda sparsification following low-rank approximation further enhances compression efficiency. Applying sparsity after low-rank approximation (30 %+Sparsity and onwards) allows for the removal of additional redundant weights, thereby refining the model’s parameter structure without significantly impacting performance. The

dynamic selection mechanism inherent in LS-PRISM ensures that only those approximations and pruning actions that do not adversely affect, and may even improve, model performance are retained. This is particularly effective in larger models like Gemma-2B, which possess higher parameter redundancy, providing greater flexibility for compression without compromising accuracy.

Additionally, results obtained using our KMIF-based dynamic rank selection (KMIF) further highlight the effectiveness of the dynamic rank selection approach. Specifically, KMIF achieves a notable balance, reducing the model size while maintaining high accuracy scores of 62.29 % on BoolQ and 56.31 % on RTE. When combined with Wanda sparsification (KMIF+Sparsity), the method maintains competitive performance and achieves additional sparsity-induced efficiency, reinforcing the advantages of dynamically selecting the truncation rank based on singular value distributions rather than fixed removal rates. These outcomes demonstrate that KMIF effectively captures layer-specific redundancy, thus optimizing model compression with minimal performance degradation.

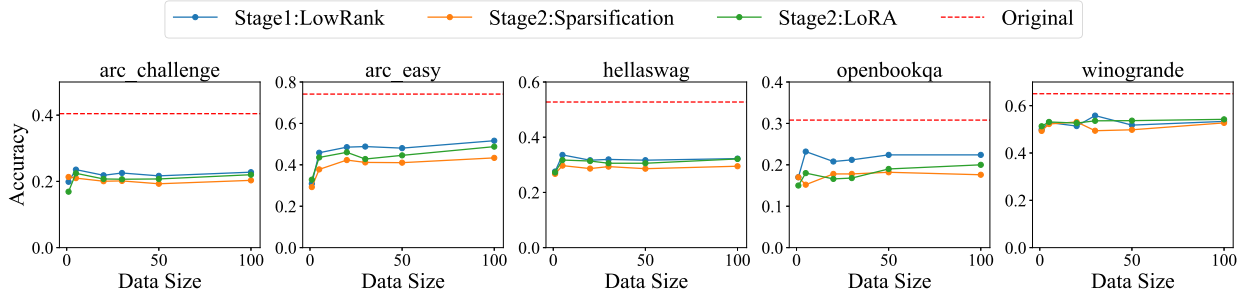


Fig. 4. Accuracy vs. Calibration Data Size for the Gemma-2B model.

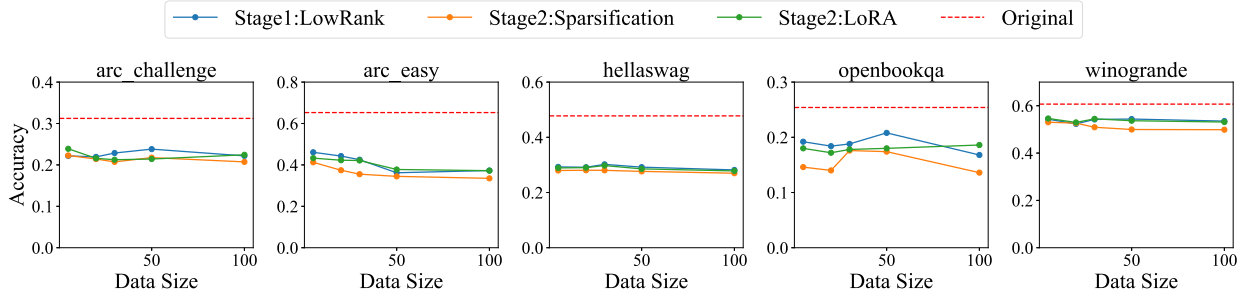


Fig. 5. Accuracy vs. Calibration Data Size for the Phi-1.5 model.

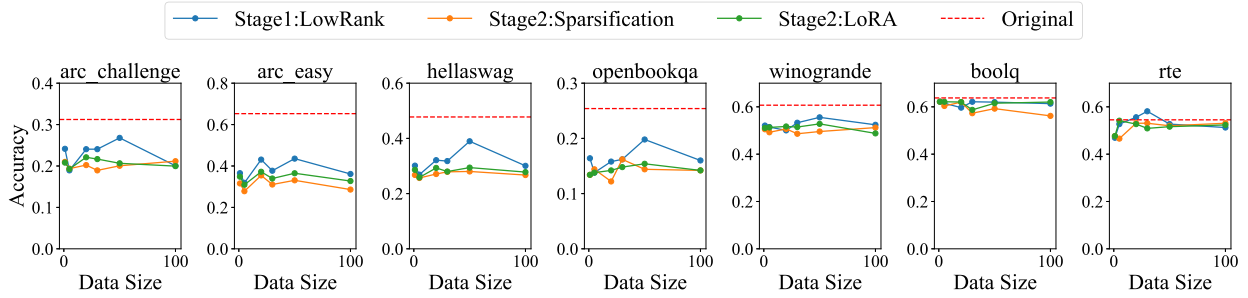


Fig. 6. Accuracy vs. Calibration Data Size for the LLaMA model.

#### 4.3. Additional experimental analysis: model compression performance across multiple models and tasks

To further elucidate the impact of calibration dataset size on model compression performance, we conducted additional in-depth analyses visualized in Figs. 4–7. These results systematically illustrate the impact of calibration data size on accuracy for the Gemma-2B, Phi-1.5, and LLaMA3.2-1B models, with additional analyses of compressed model size and sparsity specifically provided for the LLaMA3.2-1B model.

Fig. 4 shows that the accuracy of the Gemma-2B model steadily improves as the calibration data size increases from 1 to 20 samples, but beyond this point, further increments in data size yield only marginal gains. This observation highlights an optimal calibration size around 20 to 30 samples, balancing model accuracy retention with compression efficiency.

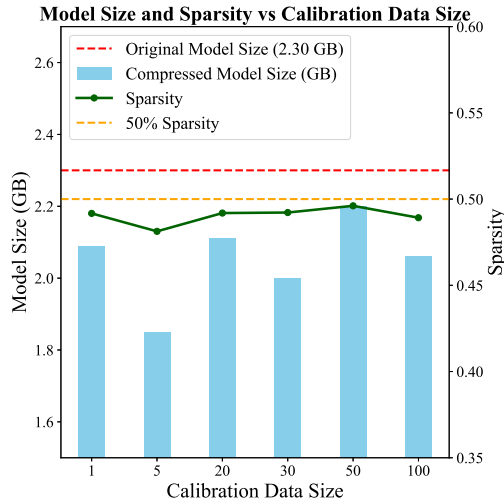
A similar trend is evident in Fig. 5 for the smaller-scale Phi-1.5 model, although the sensitivity to calibration dataset size is more pronounced, emphasizing the importance of carefully selecting an appropriate calibration dataset size.

Fig. 6 extends this analysis to the LLaMA model, again demonstrating a similar diminishing-return pattern. A moderate calibration data size

provides a balanced compromise between accuracy and computational overhead, confirming consistency across different model architectures and scales.

To comprehensively illustrate the joint impact of calibration data size on both model size and sparsity, Fig. 7 provides a dual-axis visualization specifically for the LLaMA3.2-1B model. The bar plots indicate the compressed model sizes, clearly showing consistent reductions relative to the original model size (2.30 GB, indicated by the red dashed baseline). Simultaneously, the sparsity trend, represented by the green line plot, remains stable around approximately 49%, closely aligned with the reference sparsity level of 50% (orange dashed line). This visualization effectively demonstrates that, despite variations in calibration dataset size, the compression strategy reliably maintains a stable sparsity ratio while achieving significant model size reduction.

Overall, this comprehensive visual analysis underscores the crucial role of calibration dataset size in optimizing the delicate balance between compression efficiency and task-specific accuracy across various model architectures, thereby guiding practical decisions in applying two-stage compression techniques to large-scale language models.



**Fig. 7.** Model Size and Sparsity vs. Calibration Data Size for the LLaMA3.2-1B model. The bar chart represents compressed model size, with the red dashed line indicating the original model size (2.30 GB). The green line with markers shows the sparsity trend, and the orange dashed line represents a reference sparsity level of 50%.

#### 4.4. Analysis of task-specific performance variations

In our experiments, we observed variations in the effectiveness of LS-PRISM across different evaluation tasks. Specifically, while LS-PRISM achieved minimal accuracy loss on tasks such as BoolQ and RTE, we observed relatively larger performance drops on tasks demanding complex reasoning or extensive commonsense knowledge, including ARC-easy, ARC-Challenge, HellaSwag, Winogrande, and OpenBookQA.

Unlike uniform compression strategies, LS-PRISM employs a dynamic, layer-selective approach that individually applies low-rank approximations based on each matrix's impact on accuracy and loss, assessed using a small calibration set of less than 100 samples from the CounterFact dataset. Although this approach effectively preserves performance for certain tasks, the relatively limited number and scope of calibration samples from the CounterFact dataset may not fully represent the complexity or diversity required by reasoning-intensive tasks. Tasks like ARC, HellaSwag, Winogrande, and OpenBookQA inherently involve subtle and distributed commonsense knowledge, making them more sensitive to compression-induced information loss.

Additionally, for efficiency reasons, our LoRA fine-tuning phase utilized only 1000 samples from the commonsense-restored commonsense\_170k dataset. Although this approach effectively restored some general commonsense capabilities, the limited dataset size may have been insufficient for adequately recovering the specialized, task-specific knowledge essential for benchmarks such as ARC or OpenBookQA. Consequently, these tasks exhibited more pronounced accuracy degradation.

To address these limitations, future iterations of LS-PRISM could incorporate larger or task-specific calibration datasets during layer selection to better capture critical knowledge representations for reasoning-intensive tasks. Additionally, more targeted fine-tuning on datasets explicitly designed for tasks such as ARC and OpenBookQA, would further enhance LS-PRISM's versatility and robustness.

#### 4.5. Discussion

The experimental results presented in Tables 1–5 collectively demonstrate the efficacy and versatility of LS-PRISM in compressing Large Language Models (LLMs) while maintaining or enhancing their performance. For the Gemma-2B model, an optimal calibration data size of 20

to 100 samples ensures significant parameter and size reductions with minimal accuracy loss, highlighting the importance of adequate calibration data in guiding effective low-rank approximations. The integration of Wanda sparsification with LoRA fine-tuning further enhances compression efficiency by inducing additional sparsity and recovering performance losses, as evidenced by the improved accuracy metrics in both BoolQ and RTE tasks. Additionally, the analysis of Singular Value Removal Rates reveals that higher removal rates (up to 90%) can achieve substantial compression without compromising, and sometimes even improving, model performance, thanks to the model's inherent parameter redundancy. However, excessively high removal rates lead to performance degradation, emphasizing the need for a balanced approach. In contrast, the smaller Phi-1.5 model, with its lower parameter redundancy, exhibits more pronounced performance drops under aggressive compression, particularly in complex tasks like RTE, when using limited calibration data. This underscores that while LS-PRISM is highly effective for larger models, careful calibration and moderate compression strategies are essential for models to preserve their functional integrity. For the LLaMA3.2-1B model, the results further validate the versatility of LS-PRISM, highlighting stable sparsity levels and consistent reductions in model size across varying calibration data sizes. Notably, the compressed models consistently achieved around 49% sparsity and remained significantly smaller than the original model size, underscoring the approach's effectiveness in systematically managing parameter redundancy without compromising the model's integrity. This stability reinforces the importance of selecting appropriate calibration data sizes to optimize both compression efficiency and model performance across diverse evaluation settings.

## 5. Conclusion

In this paper, we have introduced **LS-PRISM**, an innovative method for compressing Large Language Models (LLMs) through a dynamic, layer-selective approach that combines dynamic low-rank approximation and sparsification. LS-PRISM effectively reduces model size and computational demands while preserving, and in some instances enhancing, model performance. By strategically selecting weight matrices for low-rank approximation based on their influence on accuracy and loss, and subsequently applying unstructured pruning using the Wanda method, LS-PRISM achieves significant compression with minimal trade-offs in performance. Our extensive experiments on both the Gemma-2B and Phi-1.5 models demonstrate LS-PRISM's robustness and versatility across different model scales and configurations. LS-PRISM excels in compressing larger models with higher parameter redundancy, achieving up to a 12% reduction in parameters and model size for Gemma-2B without compromising accuracy. While smaller models like Phi-1.5 and LLaMA3.2-1B exhibit more sensitivity to compression, LS-PRISM still provides effective reductions by carefully balancing calibration data size and compression rates. The integration of LoRA fine-tuning further enhances performance recovery, underscoring the method's adaptability. Overall, LS-PRISM offers a scalable and efficient solution for deploying LLMs in resource-constrained devices. As future work, we aim to extend LS-PRISM beyond textual models to include multimodal architectures such as VLMs and MLLMs, exploring its ability to compress models while preserving cross-modal performance across heterogeneous representation spaces.

#### CRedit authorship contribution statement

**Renshuai Tao:** Methodology, Conceptualization, Writing – original draft, Formal analysis, Project administration, Data curation; **Hairong Chen:** Investigation, Software, Methodology; **Yuzhe Guo:** Data curation, Formal analysis; **Jiakai Wang:** Visualization, Validation, Writing – review & editing; **Boying Wang:** Visualization, Validation, Writing – review & editing; **Rongrong Ni:** Methodology, Supervision, Funding

acquisition, Project administration; **Yao Zhao**: Project administration, Conceptualization, Methodology, Supervision, Funding acquisition.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Acknowledgment

This work was supported by Beijing NSF (Nos. L242021, 4222014), National NSF of China (Nos. U24B20179, 62336001), and the Talent Fund of Beijing Jiaotong University (Nos. 2024XKRC047, 2024XKRC011).

## References

- Ayinde, B. O., Inanc, T., & Zurada, J. M. (2019). Redundant feature pruning for accelerated inference in deep neural networks. *Neural Networks*, 118, 148–158.
- Bentivogli, L., Clark, P., Dagan, I., & Giampiccolo, D. (2009). The fifth PASCAL recognizing textual entailment challenge. *TAC*, 7(8), 1.
- Brown, T. B. (2020). Language models are few-shot learners. *arXiv preprint arXiv:2005.14165*.
- Chen, H., Lv, C., Ding, L., Qin, H., Zhou, X., Ding, Y., Liu, X., Zhang, M., Guo, J., Liu, X. et al. (2024). DB-LLM: Accurate dual-binarization for efficient llms. *arXiv preprint arXiv:2402.11960*.
- Clark, C., Lee, K., Chang, M.-W., Kwiatkowski, T., Collins, M., & Toutanova, K. (2019). Boolq: Exploring the surprising difficulty of natural yes/no questions. *arXiv preprint arXiv:1905.10044*.
- Clark, P., Cowhey, I., Etzioni, O., Khot, T., Sabharwal, A., Schoenick, C., & Tafjord, O. (2018). Think you have solved question answering? Try ARC, the AI2 reasoning challenge. *arXiv:1803.05457v1*.
- Devlin, J. (2018). Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*.
- Falini, A. (2022). A review on the selection criteria for the truncated SVD in data science applications. *Journal of Computational Mathematics and Data Science*, 5, 100064.
- Feng, L., Zhang, X.-L., & Li, X. (2024). Eliminating quantization errors in classification-based sound source localization. *Neural Networks*, 181, 106679.
- Franter, E., & Alistarh, D. (2023). Sparsegpt: Massive language models can be accurately pruned in one-shot. In *International conference on machine learning* (pp. 10323–10337). PMLR.
- Gao, C., Liu, S., Chen, J., Wang, L., Wu, Q., Li, B., & Tian, Q. (2023). Room-object entity prompting and reasoning for embodied referring expression. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 46, 994–1010.
- Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Vaughan, A. et al. (2024). The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.
- Gupta, S., Agrawal, A., Gopalakrishnan, K., & Narayanan, P. (2015). Deep learning with limited numerical precision. In *International conference on machine learning* (pp. 1737–1746). PMLR.
- Han, S., Mao, H., & Dally, W. J. (2015). Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding. *arXiv preprint arXiv:1510.00149*.
- Hinton, G. (2015). Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*.
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2021). Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*.
- Huang, W., Liu, Y., Qin, H., Li, Y., Zhang, S., Liu, X., Magno, M., & Qi, X. (2024a). BiLLM: Pushing the limit of post-training quantization for LLMs. In *International conference on machine learning*.
- Huang, W., Qin, H., Liu, Y., Li, Y., Liu, X., Benini, L., Magno, M., & Qi, X. (2024b). Slim-LLM: Saliency-driven mixed-precision quantization for large language models. *arXiv preprint arXiv:2405.14917*.
- Jaderberg, M., Vedaldi, A., & Zisserman, A. (2014). Speeding up convolutional neural networks with low rank expansions. *arXiv preprint arXiv:1405.3866*.
- Kirtas, M., Oikonomou, A., Passalis, N., Mourgiyas-Alexandris, G., Moralis-Pegios, M., Pleros, N., & Tefas, A. (2022). Quantization-aware training for low precision photonic neural networks. *Neural Networks*, 155, 561–573.
- Li, Y., Bubeck, S., Eldan, R., Del Giorno, A., Gunasekar, S., & Lee, Y. T. (2023a). Textbooks are all you need II: Phi-1.5 technical report. *arXiv preprint arXiv:2309.05463*.
- Li, Z., Usman, M., Tao, R., Xia, P., Wang, C., Chen, H., & Li, B. (2023b). A systematic survey of regularization and normalization in GANs. *ACM Computing Surveys*, 55(11), 1–37.
- Liu, Y., Shi, D., & Lin, H. (2024a). Few-shot learning with representative global prototype. *Neural Networks*, 180, 106600.
- Liu, Z., Du, B., Ye, J., Wen, X., & Sun, L. (2024b). An NCDE-based framework for universal representation learning of time series. *IJCAI*, 4623–4633.
- Ma, Y., Li, H., Zhang, Z., Guo, J., Zhang, S., Gong, R., & Liu, X. (2023). Annealing-based label-transfer learning for open world object detection. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 11454–11463).
- Meng, K., Bau, D., Andonian, A., & Belinkov, Y. (2022). Locating and editing factual associations in GPT. *Advances in Neural Information Processing Systems*, 35, 17359–17372.
- Merity, S., Xiong, C., Bradbury, J., & Socher, R. (2016). Pointer sentinel mixture models. *arXiv preprint arXiv:1609.07843*.
- Mi, C., Xie, L., & Zhang, Y. (2022). Improving data augmentation for low resource speech-to-text translation with diverse paraphrasing. *Neural Networks*, 148, 194–205.
- Mihaylov, T., Clark, P., Khot, T., & Sabharwal, A. (2018). Can a suit of armor conduct electricity? A new dataset for open book question answering. In *Emnlp*.
- Pan, B., Li, C., & Che, H. (2023). Nonconvex low-rank tensor approximation with graph and consistent regularizations for multi-view subspace learning. *Neural Networks*, 161, 638–658.
- Poyatos, J., Molina, D., Martinez, A. D., Del Ser, J., & Herrera, F. (2023). Evoprunedeeptl: An evolutionary pruning model for transfer learning based deep neural networks. *Neural Networks*, 158, 59–82.
- Qi, B., Sun, H., Zhang, H., Zhao, R., & Gao, X. (2024a). Modularizing while training: A new paradigm for modularizing DNN models. In *Proceedings of the 46th IEEE/ACM international conference on software engineering* (pp. 1–12).
- Qi, P., Sun, Y., Yao, M., & Tao, D. (2024b). Kemos: A knowledge-enhanced multi-modal summarizing framework for chinese online meetings. *Neural Networks*, 178, 106417.
- Qin, H., Ding, Y., Zhang, M., Qinghua, Y., Liu, A., Dang, Q., Liu, Z., & Liu, X. (2022). BiBERT: Accurate fully binarized BERT. In *International conference on learning representations*.
- Qin, H., Gong, R., Liu, X., Shen, M., Wei, Z., Yu, F., & Song, J. (2020). Forward and backward information retention for accurate binary neural networks. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 2250–2259).
- Qin, H., Ma, X., Zheng, X., Li, X., Zhang, Y., Liu, S., Luo, J., Liu, X., & Magno, M. (2024). Accurate loRA-finetuning quantization of LLMs via information retention. In *International conference on machine learning*.
- Qiu, H., Ning, M., Song, Z., Fang, W., Chen, Y., Sun, T., Ma, Z., Yuan, L., & Tian, Y. (2024). Self-architectural knowledge distillation for spiking neural networks. *Neural Networks*, 178, 106475.
- Qiu, Y., Zhou, G., Zeng, J., Zhao, Q., & Xie, S. (2022). Imbalanced low-rank tensor completion via latent matrix factorization. *Neural Networks*, 155, 369–382.
- Sakaguchi, K., et al., (2019). Winogrande: An adversarial winograd schema challenge at scale.
- Sharma, P., Ash, J. T., & Misra, D. (2023). The truth is in there: Improving reasoning in language models with layer-selective rank reduction. *arXiv preprint arXiv:2312.13558*.
- Sun, M., Liu, Z., Bair, A., & Kolter, J. Z. (2023). A simple and effective pruning approach for large language models. *arXiv preprint arXiv:2306.11695*.
- Sun, Q., Li, J., Yuan, H., Fu, X., Peng, H., Ji, C., Li, Q., & Yu, P. S. (2022). Position-aware structure learning for graph topology-imbalance by relieving under-reaching and over-squashing. In *Proceedings of the 31st ACM international conference on information & knowledge management* (pp. 1848–1857).
- Tao, R., Li, H., Wang, T., Wei, Y., Ding, Y., Jin, B., Zhi, H., Liu, X., & Liu, A. (2022). Exploring endogenous shift for cross-domain detection: A large-scale benchmark and perturbation suppression network. In *2022 IEEE/CVF conference on computer vision and pattern recognition (CVPR)* (pp. 21157–21167). IEEE.
- Team, G., Mesnard, T., Hardin, C., Dadashi, R., Bhupatiraju, S., Pathak, S., Sifre, L., Riviere, M., Kale, M. S., Love, J. et al. (2024a). Gemma: Open models based on gemini research and technology. *arXiv preprint arXiv:2403.08295*.
- Team, G., Riviere, M., Pathak, S., Sessa, P. G., Hardin, C., Bhupatiraju, S., Hussenot, L., Mesnard, T., Shahrirari, B., Ramé, A. et al. (2024b). Gemma 2: Improving open language models at a practical size. *arXiv preprint arXiv:2408.00118*.
- Wang, A. (2018). Glue: A multi-task benchmark and analysis platform for natural language understanding. *arXiv preprint arXiv:1804.07461*.
- Wang, X., Wang, S., Tang, C., Zhu, L., Jiang, B., Tian, Y., & Tang, J. (2024a). Event stream-based visual object tracking: A high-resolution benchmark dataset and a novel baseline. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 19248–19257).
- Wang, X., Zheng, Y., Wan, Z., & Zhang, M. (2024b). SVD-LLM: Truncation-aware singular value decomposition for large language model compression. *arXiv preprint arXiv:2403.07378*.
- Wang, Z., Wohlwend, J., & Lei, T. (2019). Structured pruning of large language models. *arXiv preprint arXiv:1910.04732*.
- Wu, S., Yu, J., Chen, J., & Zhou, W. (2024). Generative commonsense knowledge sub-graph retrieval for open-domain dialogue response generation. *Neural Networks*, 180, 106666.
- Xian, Y., Lampert, C. H., Schiele, B., & Akata, Z. (2018). Zero-shot learning-a comprehensive evaluation of the good, the bad and the ugly. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 41(9), 2251–2265.
- Yuan, Z., Shang, Y., Song, Y., Wu, Q., Yan, Y., & Sun, G. (2023). Asvd: Activation-aware singular value decomposition for compressing large language models. *arXiv preprint arXiv:2312.05821*.
- Zellers, R., Holtzman, A., Bisk, Y., Farhadi, A., & Choi, Y. (2019). Hellaswag: Can a machine really finish your sentence? In *Proceedings of the 57th annual meeting of the association for computational linguistics*.
- Zhou, C., Wang, H., Zhou, S., Yu, Z., Bandara, D., & Bu, J. (2023). Hierarchical knowledge propagation and distillation for few-shot learning. *Neural Networks*, 167, 615–625.
- Zhu, X., Li, J., Liu, Y., Ma, C., & Wang, W. (2024). Distilling mathematical reasoning capabilities into small language models. *Neural Networks*, 179, 106594.